

Arquitetura de Software e Organização de Código

Diretrizes, padrões de mercado e boas práticas para a construção de sistemas escaláveis e sustentáveis

1. Introdução à Organização de Código

Em projetos de software pequenos, colocar quase toda a lógica em poucos arquivos pode parecer rápido e inofensivo. No entanto, conforme o sistema cresce, a falta de estrutura transforma o desenvolvimento num ambiente caótico. O código desorganizado gera o que chamamos de "*Código Espaguete*" — um emaranhado onde alterar uma funcionalidade numa ponta do sistema quebra três recursos distantes na outra.

A **Arquitetura de Software** serve para definir como os componentes de um sistema interagem, onde cada pedaço de código deve residir e como os dados fluem. O objetivo principal é alcançar duas metas fundamentais na engenharia:

- **Alta Coesão:** Cada classe, arquivo ou módulo deve fazer apenas uma coisa e fazê-la com excelência (foco total na sua responsabilidade).
- **Baixo Acoplamento:** As diferentes partes do sistema devem ser o mais independentes possível. Se precisar de trocar o banco de dados ou a interface visual, as regras de negócio cruciais não devem sofrer impactos.

2. Padrões Arquiteturais de Mercado

2.1. Arquitetura em Camadas (Layered Architecture)

É o padrão mais comum e intuitivo. O sistema é dividido horizontalmente em camadas sobrepostas, onde cada camada tem um papel estrito e geralmente só se comunica com a camada imediatamente inferior.

- **Camada de Apresentação (Presentation/UI):** Responsável por exibir as informações ao usuário e capturar as interações (ex: controladores REST, telas, views).
- **Camada de Negócio (Business/Service):** Onde residem as regras de negócio cruciais (ex: cálculos, validações, lógicas de aprovação).
- **Camada de Acesso a Dados (Data Access/Repository):** Responsável por se comunicar diretamente com o banco de dados ou serviços de persistência.

2.2. Arquitetura Limpa (Clean Architecture) & Hexagonal

Popularizada por Robert C. Martin (Uncle Bob), a Clean Architecture isola completamente as **Regras de Negócio (Entidades e Casos de Uso)** no centro absoluto da aplicação.

A Regra da Dependência

O código das camadas internas nunca pode conhecer nada sobre as camadas externas. As suas regras de negócio não sabem se o banco de dados é PostgreSQL ou Oracle, nem se a interface é um app móvel ou uma linha de comando. A comunicação com o mundo externo é feita através de **Interfaces (Ports)** e implementada por **Adaptadores (Adapters)**.

2.3. MVC (Model-View-Controller)

Muito utilizado no desenvolvimento web clássico, divide a aplicação em três componentes interligados:

- **Model (Modelo):** Gerencia os dados e a lógica central.
- **View (Visão):** A interface e apresentação visual dos dados para o usuário.
- **Controller (Controlador):** O intermediário que recebe as entradas da View, manipula o Model e atualiza a interface.

3. Estruturação Prática de Pastas e Diretórios

A teoria arquitetural reflete-se diretamente na árvore de diretórios do projeto. Abaixo, destacamos as duas abordagens mais eficientes utilizadas por equipas de engenharia:

Abordagem por Camadas Técnicas (Package by Layer)

Recomendado para projetos iniciais, de pequeno ou médio porte, organizando por afinidade tecnológica:

```
meu-projeto/  
├── src/  
│   ├── controllers/      # Captura requisições e envia as respostas (API)  
│   ├── models/          # Estrutura dos dados e entidades abstratas  
│   ├── services/        # Onde as regras de negócio e validações residem  
│   ├── repositories/    # Consultas ao banco de dados (SQL / ORM)  
│   └── config/           # Configurações de ambiente e conexões externas
```

Abordagem por Funcionalidade (Package by Feature)

Altamente recomendada para projetos de grande porte, sistemas corporativos e microsserviços. Mantém todo o contexto de um assunto agrupado no mesmo lugar:

```
meu-projeto/  
├─ src/  
│   ├── users/                # Tudo relacionado ao fluxo e domínio de Usuários  
│   │   ├── UserController.java  
│   │   ├── UserService.java  
│   │   └── UserRepository.java  
│   │  
│   ├── orders/              # Tudo relacionado ao fluxo e domínio de Pedidos  
│   │   ├── OrderController.java  
│   │   ├── OrderService.java  
│   │   └── OrderRepository.java
```

4. Boas Práticas Fundamentais

- **Siga os Princípios SOLID:** Especialmente o *Princípio da Responsabilidade Única (SRP)* — um arquivo ou classe deve ter apenas uma razão para mudar.
- **Evite as "Classes Deus" (God Classes):** Arquivos gigantescos com milhares de linhas que realizam múltiplas tarefas são difíceis de testar e manter. Fragmente a lógica em subcomponentes focados.
- **Inversão de Dependência:** Dependendo sempre de abstrações (interfaces) e não de classes concretas. Isso facilita a criação de testes unitários com objetos simulados (mocks).

5. Conclusão

Não existe uma "arquitetura de código perfeita", existe a **arquitetura certa para o momento e o contexto do seu projeto**. Começar com uma estrutura em camadas simples ajuda a validar ideias e criar MVPs de forma ágil. Conforme o sistema ganha complexidade e exige maior cobertura de testes automatizados, evoluir para conceitos de Clean Architecture manterá a sustentabilidade do código a longo prazo.